



2026

#HELishSummit

Power Pages: Bring your own code!

Nick Doelman

Microsoft MVP & MCT · ReadyBMS

HELish Summit 2026 · Helsinki · 17 September



ABOUT YOUR SPEAKER

Nick Doelman

I help companies build and adopt intelligent business solutions.

Microsoft MVP (8x) | Solution Architect | AI Explorer | Content Creator | Trainer | Advisor



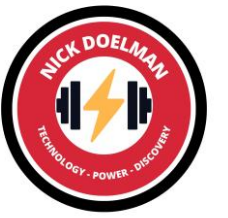
Microsoft MVP (8x)



Power Platform BOOST



Canada Powerlifting



readyxrm.blog · [ReadyBMS](#) · [Ottawa](#)

nickdoelman.com

Blog, sessions, podcast, and everything I publish. All of it lives there.



Rate this session

Feedback goes straight to me.

What you'll take away



Power Pages, fast

What the platform hands you before you write a line.



Customizing Power Pages

A real dashboard on a model-driven ticketing app.



Agentic coding

GitHub Copilot builds the site. You steer it.



Plan before you build

Where the time actually gets saved.



Where it breaks

Table permissions, blocked .js, real debugging.



Which one to pick

Configured against code first, side by side.

1

Power Pages, the short version

What the platform is, and what it already does for you.

An external facing website, built on Dataverse

Microsoft hosts it, secures it and keeps it current. You bring the data model and the pages.

Reuse what you already built

Lists and forms come straight out of your model-driven app.

Liquid and web templates

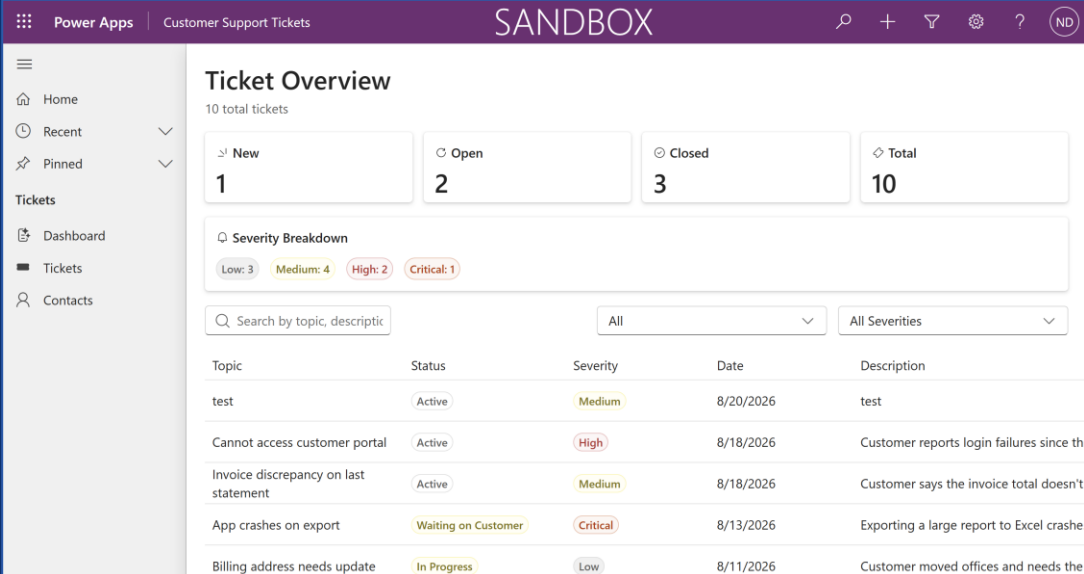
For everything configuration will not do.

Security you do not write

Web roles, page permissions and table permissions.

Sign in, handled

Entra External ID for customers. Entra ID for staff.



Topic	Status	Severity	Date	Description
test	Active	Medium	8/20/2026	test
Cannot access customer portal	Active	High	8/18/2026	Customer reports login failures since the
Invoice discrepancy on last statement	Active	Medium	8/18/2026	Customer says the invoice total doesn't
App crashes on export	Waiting on Customer	Critical	8/13/2026	Exporting a large report to Excel crashed
Billing address needs update	In Progress	Low	8/11/2026	Customer moved offices and needs the

The model-driven ticketing app this whole session is built on. Same tables, same views, same business rules. Power Pages puts a public face on it.

DEMO

A Power Pages site on an existing model-driven ticketing app

1 Start from the app

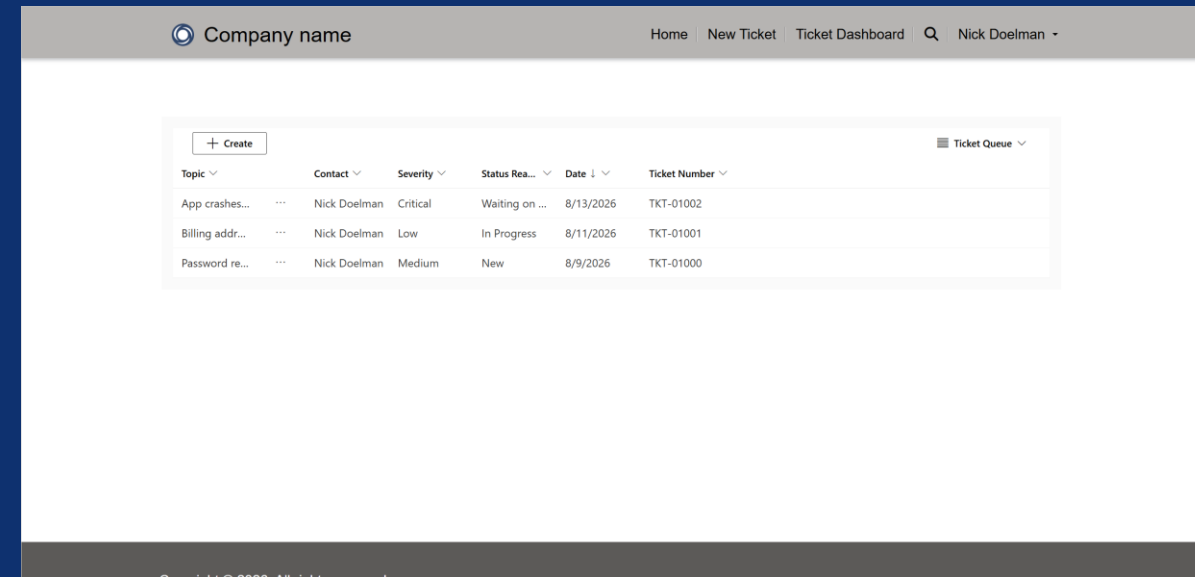
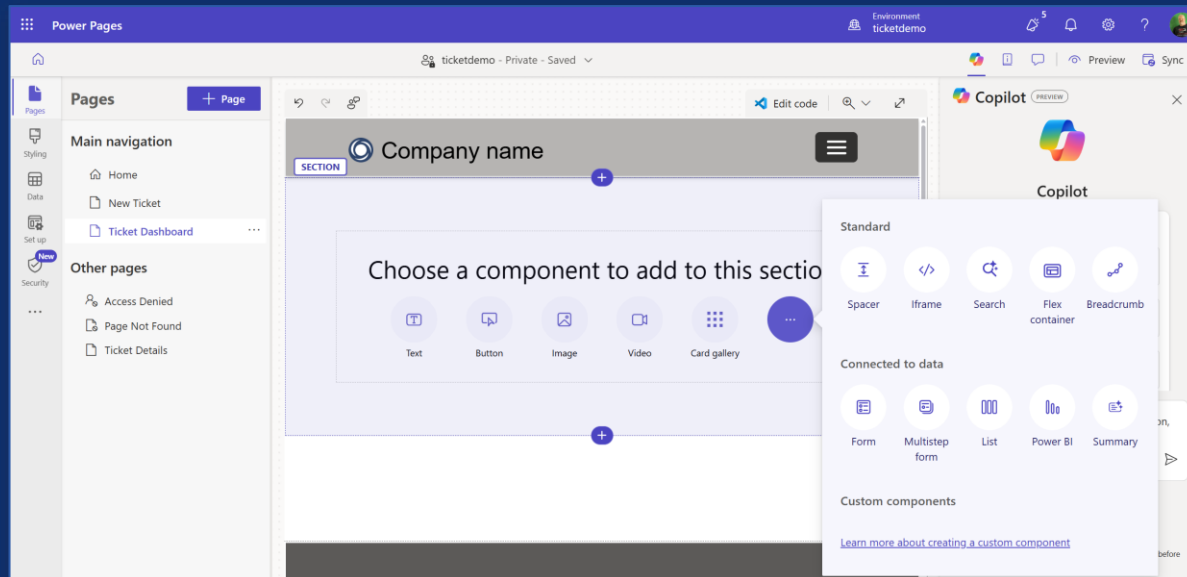
The ticket table, its views and its forms already exist.

2 Point and click

A list page and a form page, assembled in the design studio.

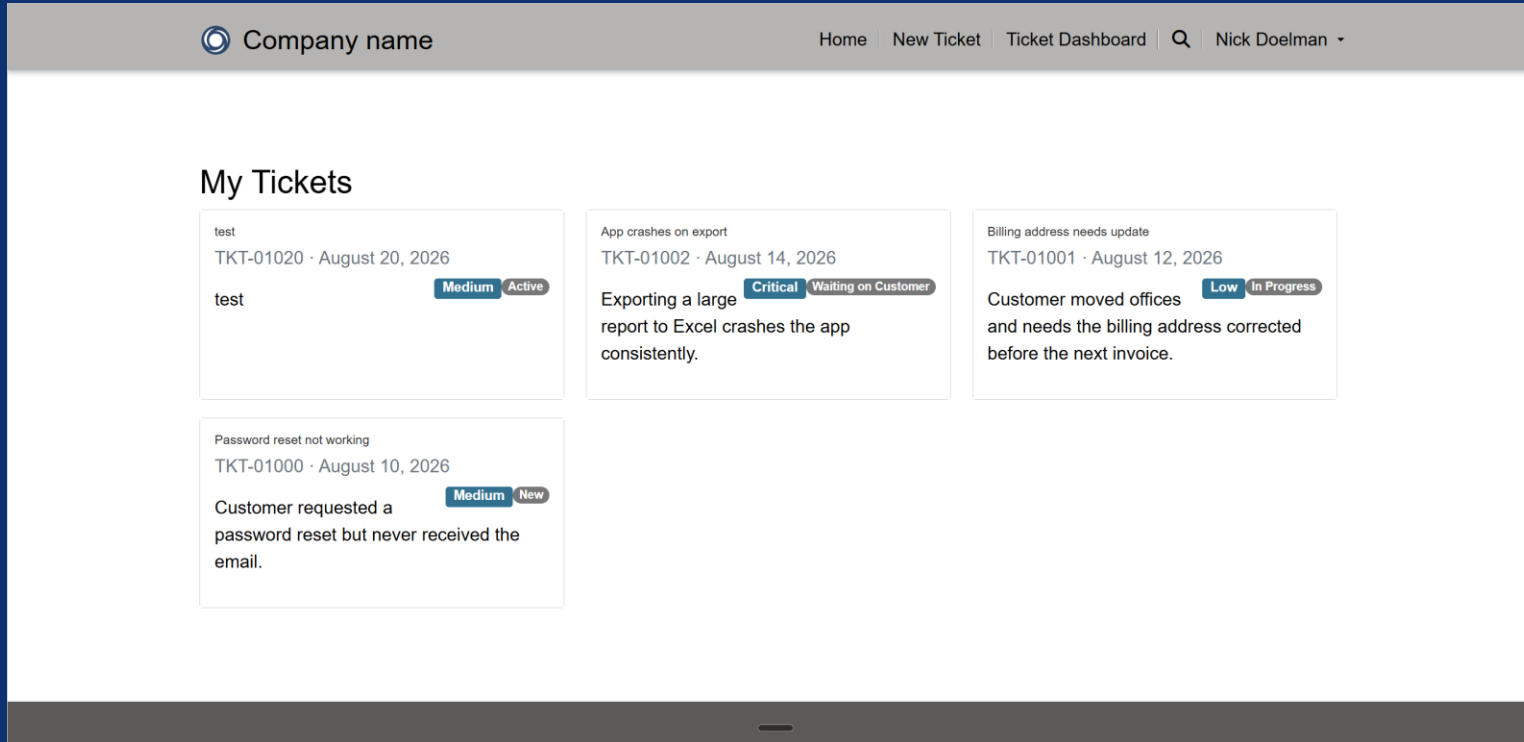
3 Ten minutes

A working, secured, signed-in site. No code yet.



WHEN CONFIGURATION IS NOT ENOUGH

A ticket dashboard, in a custom web template



Custom web template

A page the design studio cannot assemble for you.

Liquid does the work

Fetch the rows, group them, render the counts. Server side.

Still fully secured

Table permissions apply. The user only counts their own tickets.

Liquid and FetchXML, rendered before the page reaches the browser.

Lists and forms get you a long way

Then you hit the wall.

Every click is a round trip

Filter a list, reload the page. Users notice, and they compare you to everything else they use.

Interaction has a ceiling

Drag, inline edit, live counters, optimistic updates. Not what the studio is for.

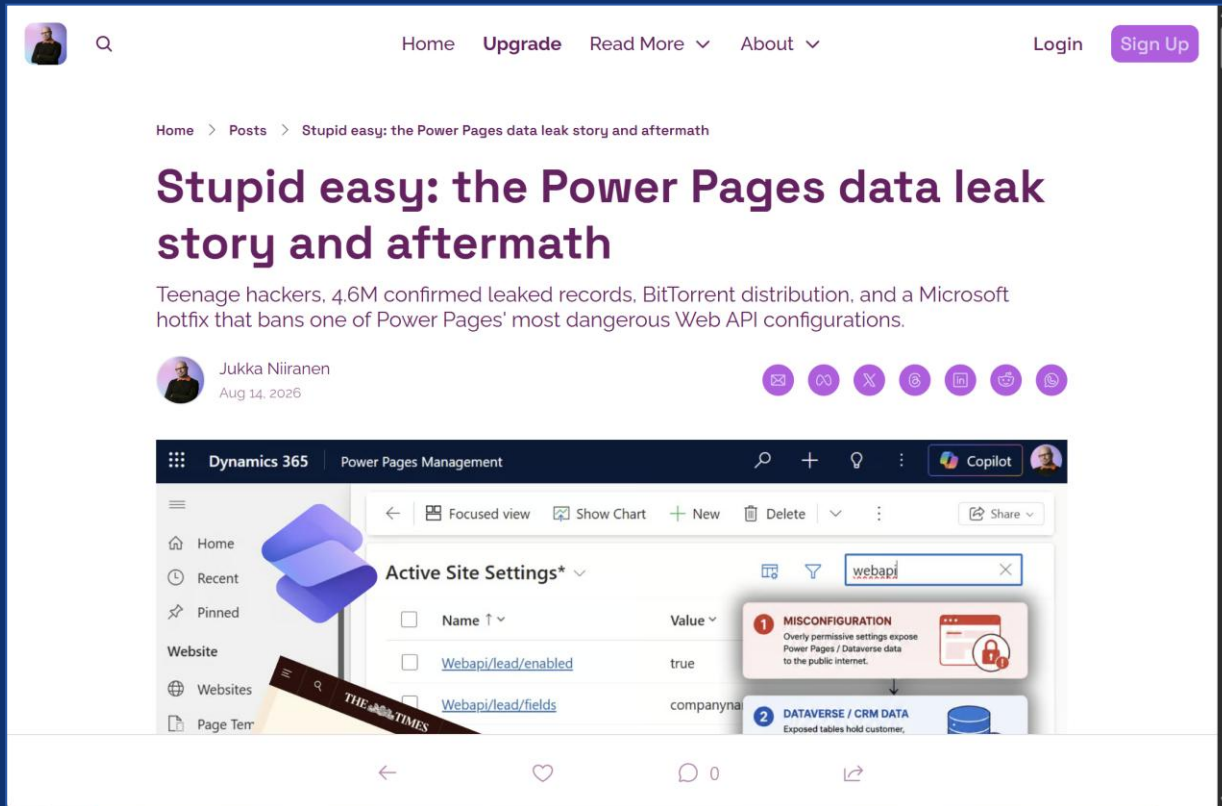
The design is the design

You can style it. Reshaping the page structure means fighting the platform.

None of that makes the configured approach wrong. It makes it a choice.

Power Pages is secure. Your configuration might not be.

There has been public reporting about Dataverse records pulled out of Power Pages sites. Nothing in it points to a platform vulnerability.



Public commentary, August 2026. Full write-up and video on readyxrm.blog.



It was configuration

A table permission scoped to anonymous visitors, on a table also enabled for the Web API. Fine apart. Not fine together.



The platform moved too

Reported alongside it: a change from Microsoft blocking one of the riskiest Web API configurations.



What to check this week

Anonymous Users table permissions. Tables enabled for the Web API. Then the security scan in the admin centre.

2

Why go code first

What you get for free, and what you are signing up for.

Power Pages hands you the hard parts for free

The things you would not want to code, build or configure from scratch.



Dataverse integration

A secure, governed data platform. No backend to stand up.



Built-in authentication

Entra External ID and work accounts, wired in, not hand rolled.



Row level security

Table permissions scope every row to the right user, server side.



Power Pages Web API

OData CRUD over your tables, contact scoped by default.



Roles and access control

Web roles and page permissions, ready out of the box.



Managed hosting and scale

Microsoft managed, global, with deployment built in.

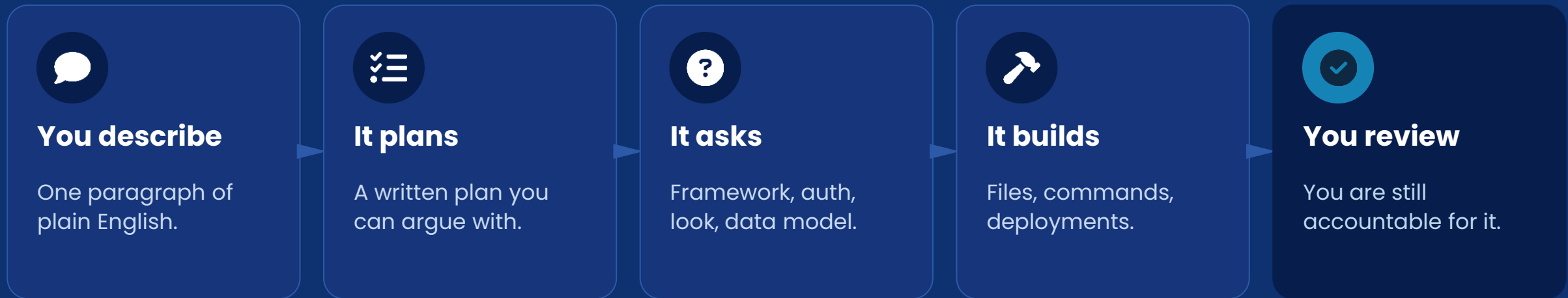
3

Agentic coding

The tools, the skills, and what the agent actually does.

Not autocomplete

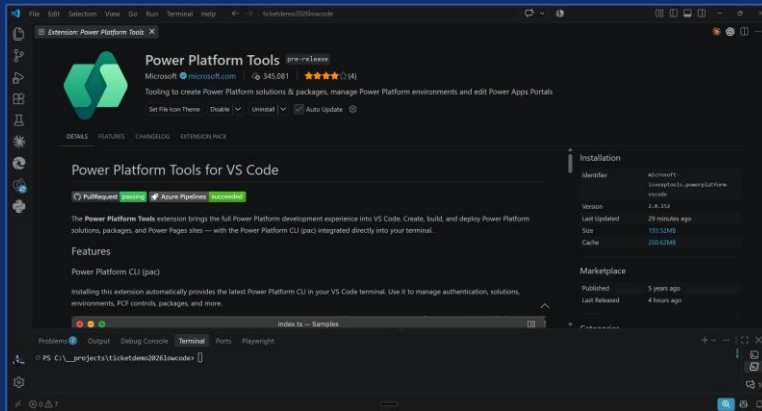
It reads your project, makes a plan, asks you questions, runs commands, edits files and checks its own work.



The loop is the point. Anything that only answers once is a chatbot, not an agent.

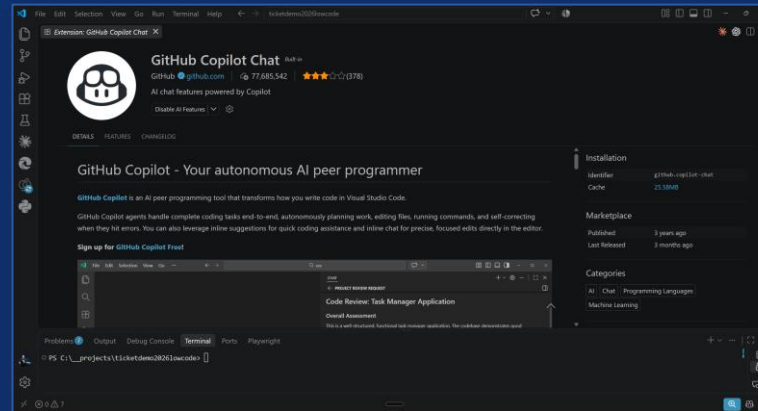
Everything runs inside Visual Studio Code

We do not open a maker portal until the very end.



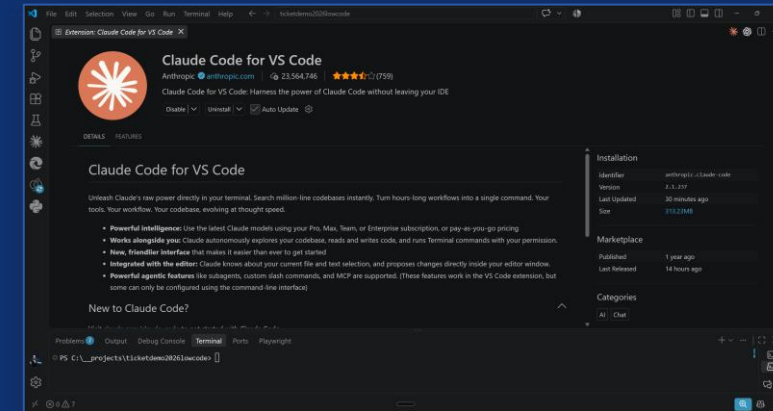
Power Platform Tools

Microsoft. Environments, solutions and the PAC CLI, in the editor.



GitHub Copilot Chat

The agent driving today. Needs a GitHub Copilot subscription.



Claude Code

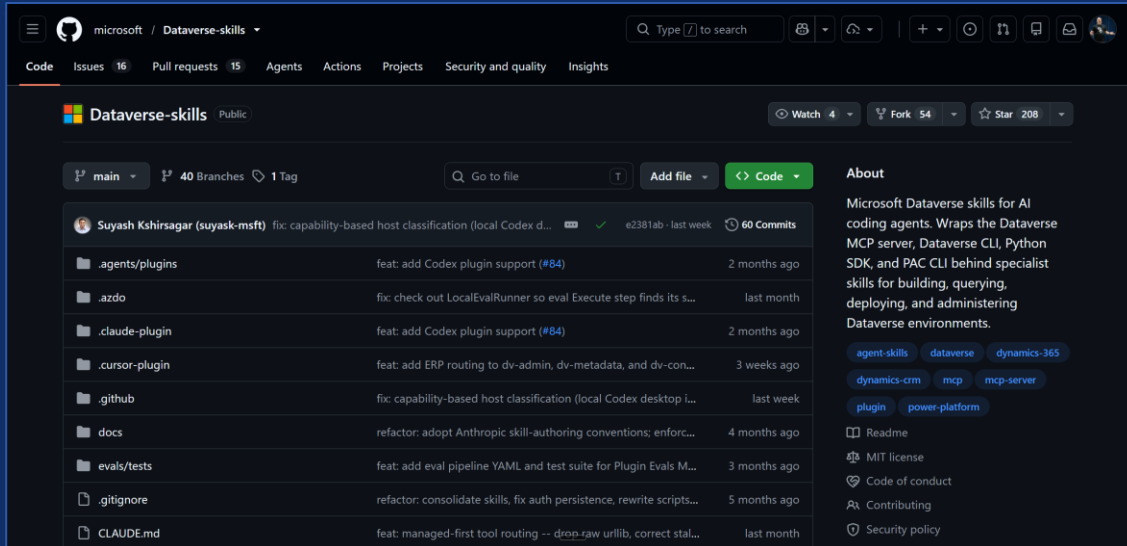
The alternative. Same shape of workflow, different subscription.

Visual Studio Code is free. Every Power Platform maker should be learning it now.

SKILLS

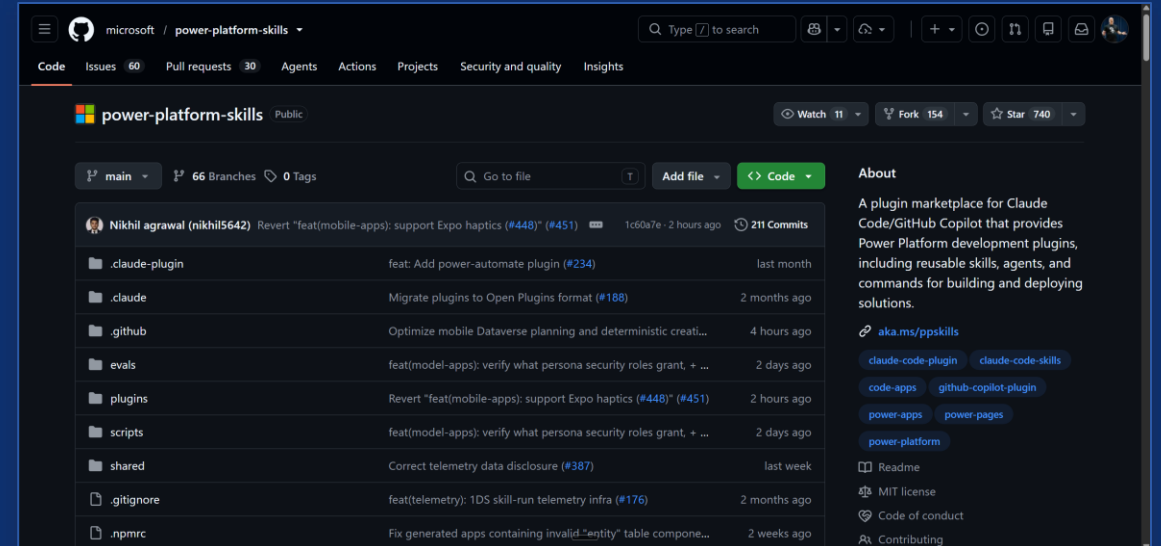
Skills are where the domain expertise lives

A general agent knows JavaScript. It does not know table permissions. Skills fix that.



Dataverse skills

Connect, query, read metadata, manage security. The agent talks to your data directly.



Power Platform skills

Includes the Power Pages skills. Site scaffolding, Web API patterns, deployment, guardrails.

github.com/microsoft/power-platform-skills · one installer script, then reload Visual Studio Code

THE TOOLS

Pick the harness you are comfortable with

GitHub Copilot, Claude Code, Codex. Not a comparison, and none of them is better. It comes down to the tool you like working in and the subscription you already pay for.

GitHub Copilot

In Visual Studio Code, on a GitHub subscription. The agent driving today.

Claude Code

Terminal or Visual Studio Code, on an Anthropic subscription. Built this same site in June.

Codex

OpenAI's agent, in the editor or the terminal. Same loop, same shape of work.

Same workflow in all three. Plan, skills, running commands, editing files, reading the errors back.

Same skills

The Dataverse and Power Platform skills install into all three.

Same result

Claude Code in June, GitHub Copilot in August. Same site.

Same decision

Use what you are comfortable with, or already pay for.

Harnesses and models change every few months. The workflow does not. That is the part worth learning.

4

Plan before you build

You can start from nothing. It is not the fastest way there.

You can build a site from nothing

Do not.

1 The agent will guess

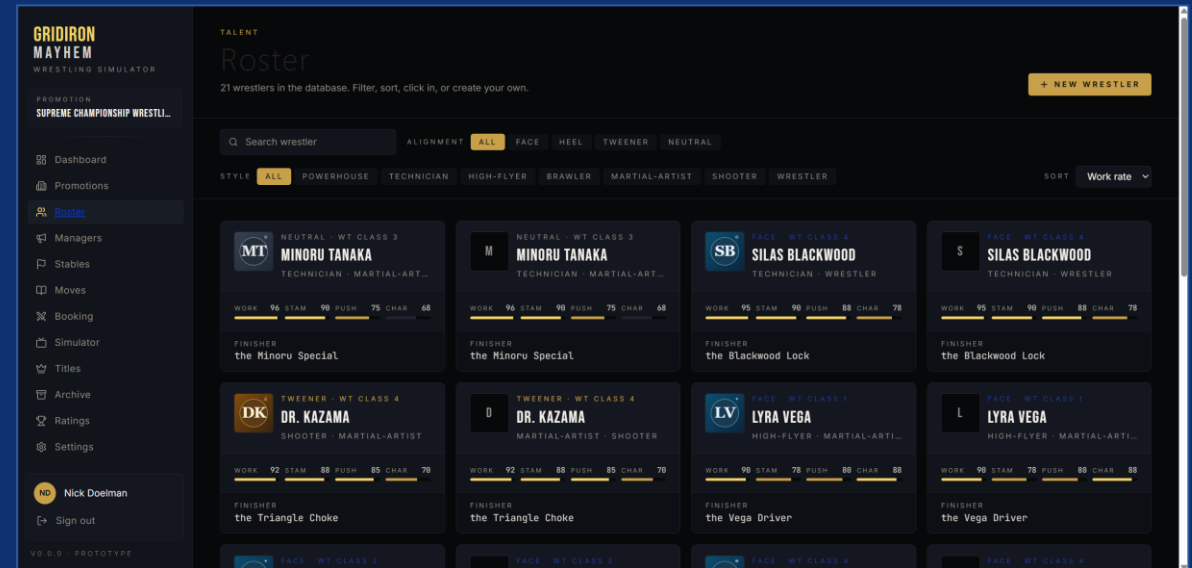
Give it no data model and it invents one. It will be plausible, and it will be wrong.

2 Rework costs more than planning

Fixing a bad schema after the components are written is the expensive part.

3 You cannot review an unscoped build

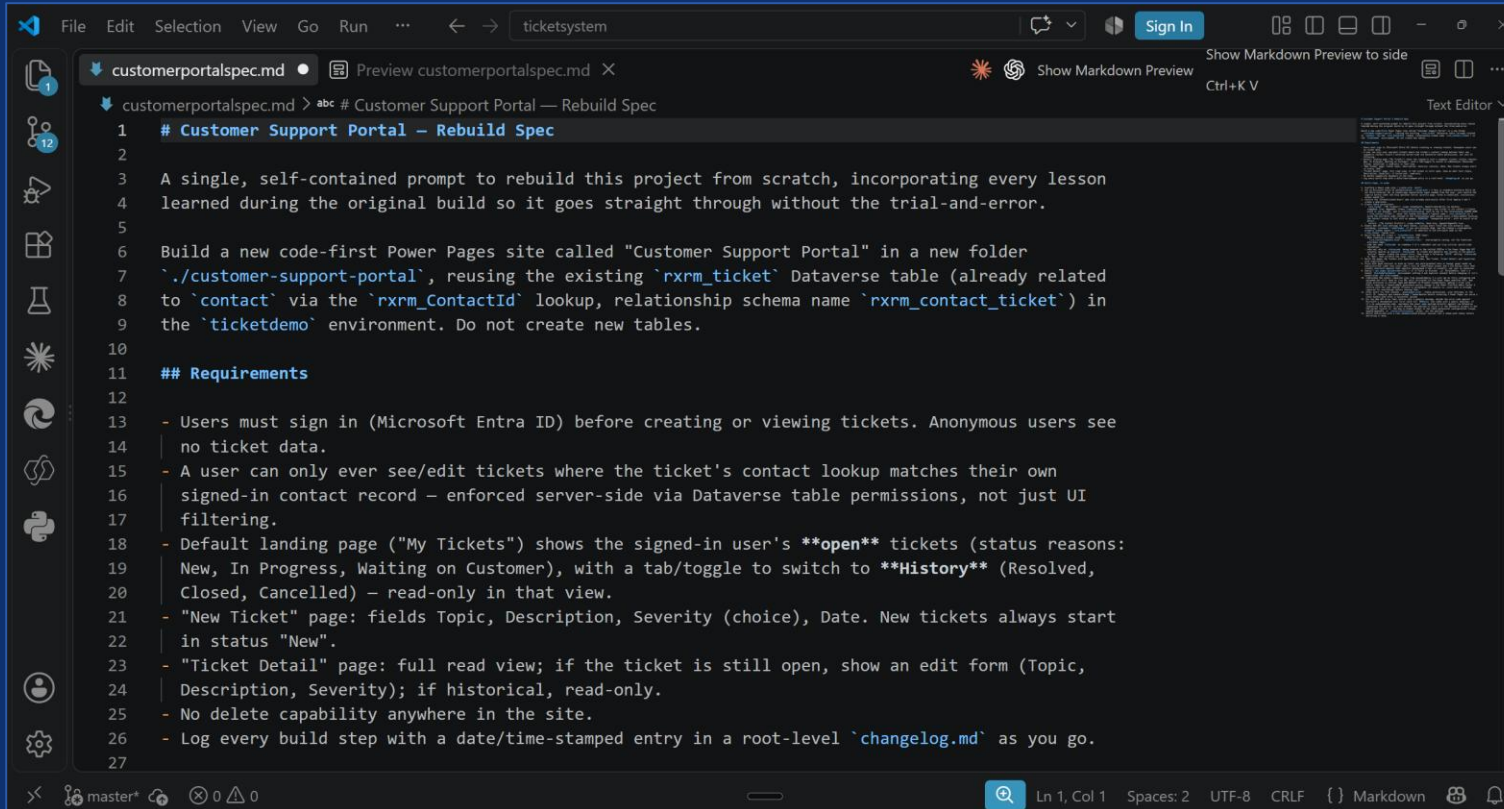
A plan you agreed to is the only thing that makes the output reviewable.



Gridiron Mayhem, vibe-coded start to finish with no plan at all. It works, and it was a lot of fun to build. It is also not something anyone else could pick up and maintain.

The time you save is not typing time. It is rework time.

Use plan mode to write the spec



```
customerportalspec.md > abc # Customer Support Portal — Rebuild Spec
1 # Customer Support Portal – Rebuild Spec
2
3 A single, self-contained prompt to rebuild this project from scratch, incorporating every lesson
4 learned during the original build so it goes straight through without the trial-and-error.
5
6 Build a new code-first Power Pages site called "Customer Support Portal" in a new folder
7 `./customer-support-portal`, reusing the existing `rxrm_ticket` Dataverse table (already related
8 to `contact` via the `rxrm_ContactId` lookup, relationship schema name `rxrm_contact_ticket`) in
9 the `ticketdemo` environment. Do not create new tables.
10
11 ## Requirements
12
13 - Users must sign in (Microsoft Entra ID) before creating or viewing tickets. Anonymous users see
14   no ticket data.
15 - A user can only ever see/edit tickets where the ticket's contact lookup matches their own
16   signed-in contact record – enforced server-side via Dataverse table permissions, not just UI
17   filtering.
18 - Default landing page ("My Tickets") shows the signed-in user's open tickets (status reasons:
19   New, In Progress, Waiting on Customer), with a tab/toggle to switch to History (Resolved,
20   Closed, Cancelled) – read-only in that view.
21 - "New Ticket" page: fields Topic, Description, Severity (choice), Date. New tickets always start
22   in status "New".
23 - "Ticket Detail" page: full read view; if the ticket is still open, show an edit form (Topic,
24   Description, Severity); if historical, read-only.
25 - No delete capability anywhere in the site.
26 - Log every build step with a date/time-stamped entry in a root-level `changelog.md` as you go.
27
```

Ten minutes writing this saves the hour you spend unpicking a build that guessed wrong.

Plan, do not build

Ask for a plan first. It reads the project and argues the approach with you.

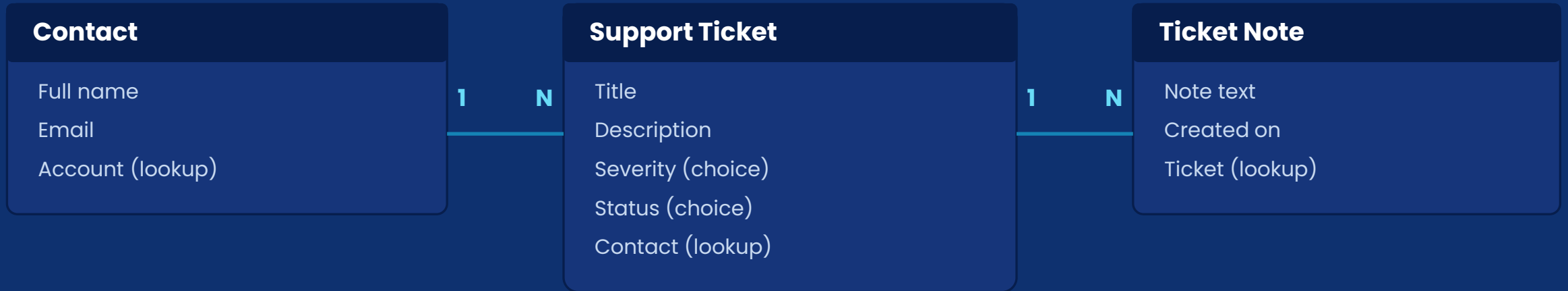
The spec is the artifact

One markdown file: tables, permissions, pages, rules, and what not to do.

Then point the build at it

Every question it would have guessed at is already answered, in your words.

Design the data model separately



Three tables, two relationships. Ten minutes on a whiteboard, before the agent writes anything.

You draw the model

Tables, columns, choices, relationships. On paper, in a diagram, anywhere but the agent.

Dataverse skills build it

Publisher, solution, tables, choice sets. Conventions captured once and reused.

Then it is a fact

Everything downstream is written against a schema you agreed to.

Write the instructions, then ask for a changelog

The screenshot shows the VS Code interface with the Copilot chat window open. The chat contains a plan for building a ticket portal. The plan includes a summary of the project, a list of tasks to be completed, and a list of questions to be asked. The chat also shows the Copilot's response to the user's request, which is to draft a plan for the new customer-facing ticket portal.

```

This is a generic scaffold skill for building Power Pages sites, but since we're in plan mode, I should adapt its discovery approach rather than follow its full interactive gate process-focusing on the specific request: an existing ticket table, portal auth, restricted visibility, and status-based views.

Since plan mode means I shouldn't mutate repo files yet, I'll skip the skill's scaffold-immediately workflow and instead ask clarifying questions about framework, auth provider, and site naming, then draft a plan.md covering the skill's phases tailored to this ticket portal before calling exit_plan_mode.

Contact mapping should work automatically since Power Pages ties authenticated users to contact records through the registration flow. I'll prepare a concise ask_user covering site name, framework, project location, and confirming the open/closed ticket status groupings match the existing ticket-overview.tsx buckets, then move on to drafting the plan.

o Asking user Waiting for response 11s

Copilot needs information.
A few quick decisions before I draft the plan for the new customer-facing ticket portal (built with the create-site workflow, using the existing rxnm_ticket table):

Site name Frontend framework Project location Which statuses count as "open" (editable, shown by default) →

Site name
Display name for the new Power Pages code site

> Customer Support Portal

enter accept · tab next · ctrl+d decline · esc cancel
  
```

/init

Copilot reads the project and writes copilot-instructions.md. Edit it. This file is the agent's memory of your conventions.

The screenshot shows the VS Code interface with the terminal window open. The terminal displays the output of the build process, including the creation of the .github directory, the creation of the copilot-instructions.md file, and the execution of the build command. The output also shows the session ID and the model used (Claude Sonnet 5).

```

This project is built entirely through Copilot CLI's model-apps skills rather than a traditional build/lint/test setup - it's a Power Platform Dataverse model-driven app defined declaratively in app-spec.json, with generated artifacts and generative pages authored as TSX using a specific runtime with caching patterns.

I'm now drafting the copilot-instructions.md to capture the project type, the single-source-of-truth role of app-spec.json, the read-only nature of .maker-workspace and RuntimeTypes.ts, the conventions for generative pages (default export, dataApi/pageInput props, module-level caching with inflight dedupe, Fluent UI), the lack of traditional build commands since changes go through the app-builder/genpage skills, and the known FetchXML primary-key issue documented separately. I'll quickly check the build-data-log files and confirm the solutions zip is just an exported package, then write the file.

MD Create copilot-instructions.md Parent directory does not exist

$ Shell Create .github directory 1 line...
New-Item -ItemType Directory -Path "C:\_projects\ticketssystem\.github" -Force | Out-Null

MD Create copilot-instructions.md +55

Created .github/copilot-instructions.md covering: this being a Power Platform/Dataverse model-driven app (no local build/lint), app-spec.json as source of truth vs. generated artifacts (model-app-plan.md, RuntimeTypes.ts, .maker-workspace/), TSX generative-page conventions (caching pattern, Fluent UI v9), and the documented FetchXML primary-key gotcha for API-created views.

No web/JS project detected here (no package.json/build scripts), so I don't see an obviously relevant MCP server to suggest beyond what's already configured. Want me to adjust any section or add coverage for something I missed?

C:\_projects\ticketssystem Session: 29.29 AIC used

add to instructions: create a changelog.md file and note updates and changes with date/time as you build the solution

@ files · # issues Claude Sonnet 5
  
```

Then add changelog.md

Tell it to log every change as it goes. When the build runs for an hour, this is how you find out what it actually did.

5

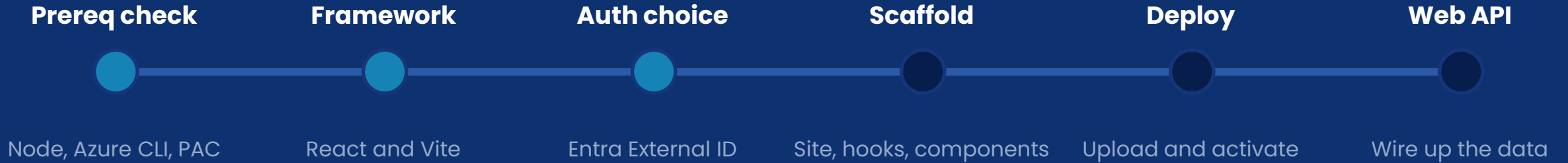
Let it run

Kick it off, then watch the checkpoints that matter.

THE BUILD

Start it, then wait

This is not a two minute demo. It runs long, and that is normal. Here is what to watch for.



Do not babysit it

The value is that you are free while it works. Go and answer email.

Read the changelog, not the scroll

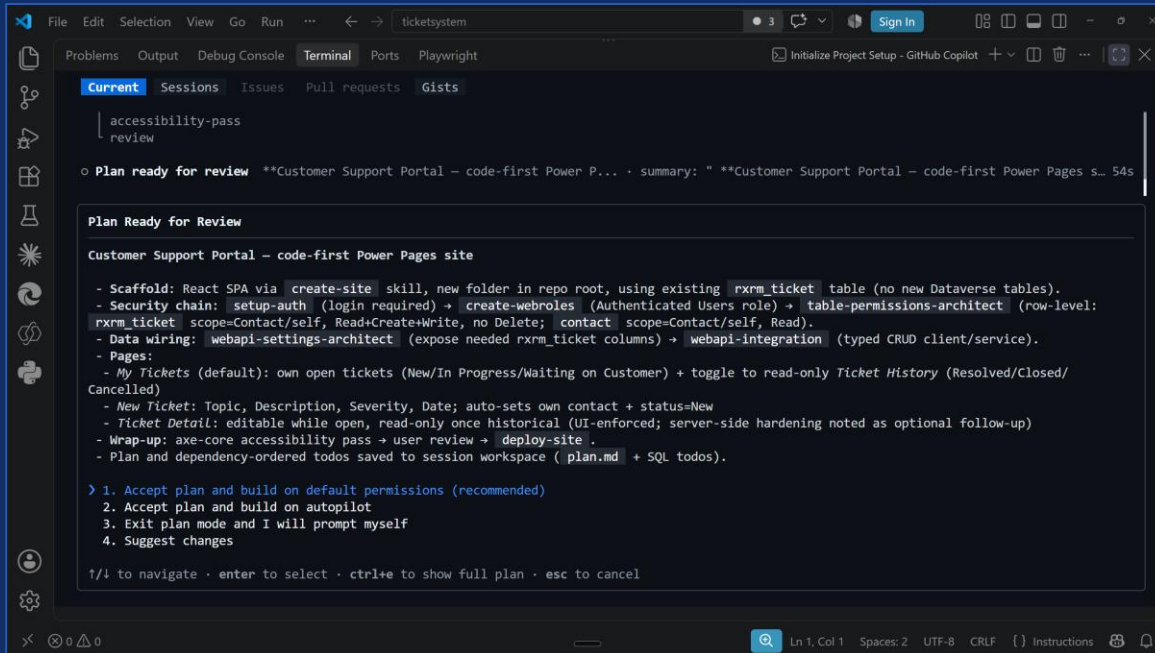
The terminal output is noise. The changelog is the summary.

It will stop and ask

Every question below is one it asked me. Answer them well.

IT WILL ASK YOU

Which framework?



The screenshot shows the VS Code interface with the Copilot chat window open. The chat displays a plan for a "Customer Support Portal - code-first Power Pages site". The plan includes details on scaffolding, security, data wiring, and pages. It also lists steps to accept the plan and build on default permissions.

```
o Plan ready for review **Customer Support Portal - code-first Power P... - summary: " **Customer Support Portal - code-first Power Pages s... 54s
```

Plan Ready for Review

Customer Support Portal - code-first Power Pages site

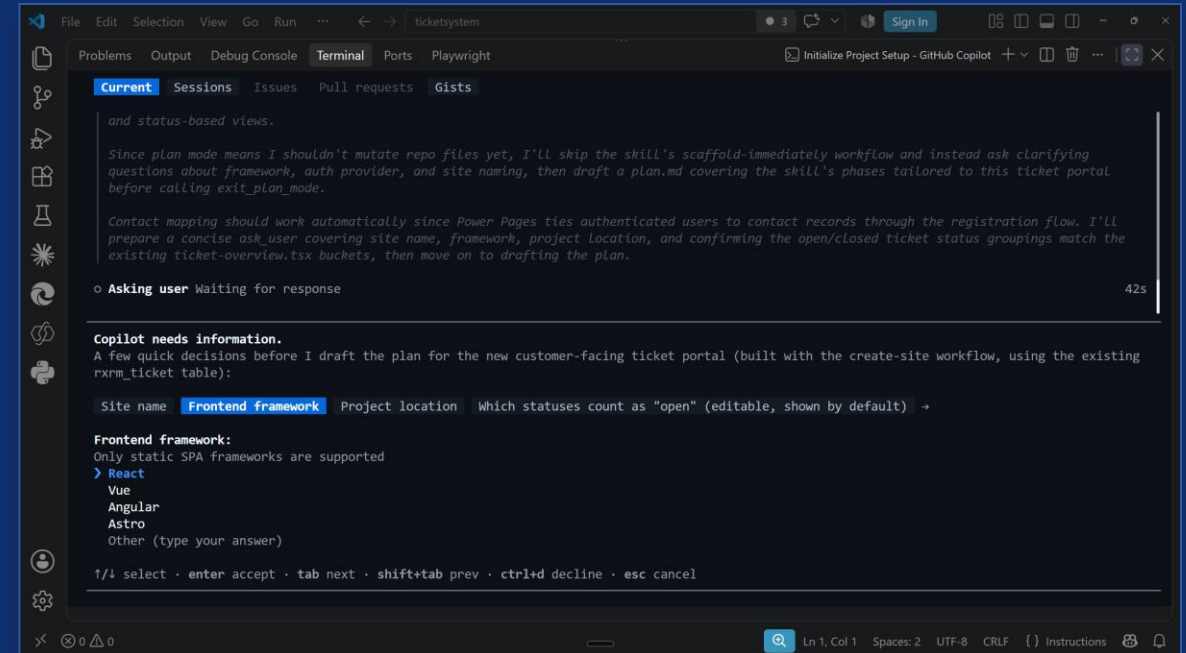
- Scaffold: React SPA via create-site skill, new folder in repo root, using existing rxrm_ticket table (no new Dataverse tables).
- Security chain: setup-auth (login required) + create-webroles (Authenticated Users role) + table-permissions-architect (row-level: rxrm_ticket scope=Contact/self, Read+Create+Write, no Delete; contact scope=Contact/self, Read).
- Data wiring: webapi-settings-architect (expose needed rxrm_ticket columns) + webapi-integration (typed CRUD client/service).
- Pages:
 - My Tickets (default): own open tickets (New/In Progress/Waiting on Customer) + toggle to read-only Ticket History (Resolved/Closed/Cancelled)
 - New Ticket: Topic, Description, Severity, Date; auto-sets own contact + status=New
 - Ticket Detail: editable while open, read-only once historical (UI-enforced; server-side hardening noted as optional follow-up)
- Wrap-up: axe-core accessibility pass + user review + deploy-site.
- Plan and dependency-ordered todos saved to session workspace (plan.md + SQL todos).

```
> 1. Accept plan and build on default permissions (recommended)
2. Accept plan and build on autopilot
3. Exit plan mode and I will prompt myself
4. Suggest changes

t/4 to navigate · enter to select · ctrl+e to show full plan · esc to cancel
```

React and Vite

The best supported path for a Power Pages code site. Take the recommendation unless you have a reason not to.



The screenshot shows the VS Code interface with the Copilot chat window open. The chat displays a plan for a "Customer Support Portal - code-first Power Pages site". The plan includes details on scaffolding, security, data wiring, and pages. It also lists steps to accept the plan and build on default permissions.

```
and status-based views.

Since plan mode means I shouldn't mutate repo files yet, I'll skip the skill's scaffold-immediately workflow and instead ask clarifying questions about framework, auth provider, and site naming, then draft a plan.md covering the skill's phases tailored to this ticket portal before calling exit_plan_mode.

Contact mapping should work automatically since Power Pages ties authenticated users to contact records through the registration flow. I'll prepare a concise ask_user covering site name, framework, project location, and confirming the open/closed ticket status groupings match the existing ticket-overview.tsx buckets, then move on to drafting the plan.

o Asking user Waiting for response 42s
```

Copilot needs information.

A few quick decisions before I draft the plan for the new customer-facing ticket portal (built with the create-site workflow, using the existing rxrm_ticket table):

Site name **Frontend framework** Project location Which statuses count as "open" (editable, shown by default) →

Frontend framework:

Only static SPA frameworks are supported

- > React
- Vue
- Angular
- Astro
- Other (type your answer)

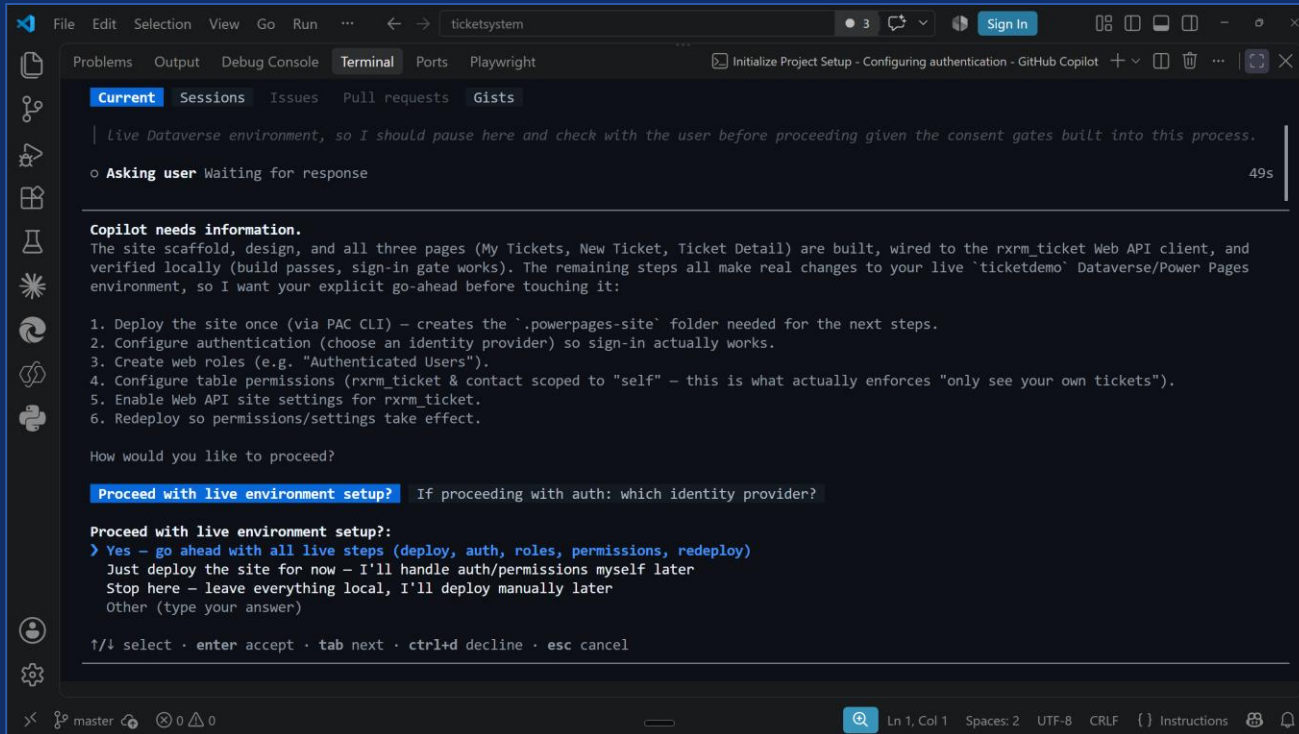
```
t/4 select · enter accept · tab next · shift+tab prev · ctrl+d decline · esc cancel
```

Prerequisites checked first

Node, Azure CLI and the PAC CLI, verified before it commits to anything.

IT WILL ASK YOU

Which identity provider?



```
File Edit Selection View Go Run ... ticketsystem
Problems Output Debug Console Terminal Ports Playwright
Current Sessions Issues Pull requests Gists
| Live Dataverse environment, so I should pause here and check with the user before proceeding given the consent gates built into this process.
o Asking user Waiting for response 49s

Copilot needs information.
The site scaffold, design, and all three pages (My Tickets, New Ticket, Ticket Detail) are built, wired to the rxrm_ticket Web API client, and
verified locally (build passes, sign-in gate works). The remaining steps all make real changes to your live `ticketdemo` Dataverse/Power Pages
environment, so I want your explicit go-ahead before touching it:

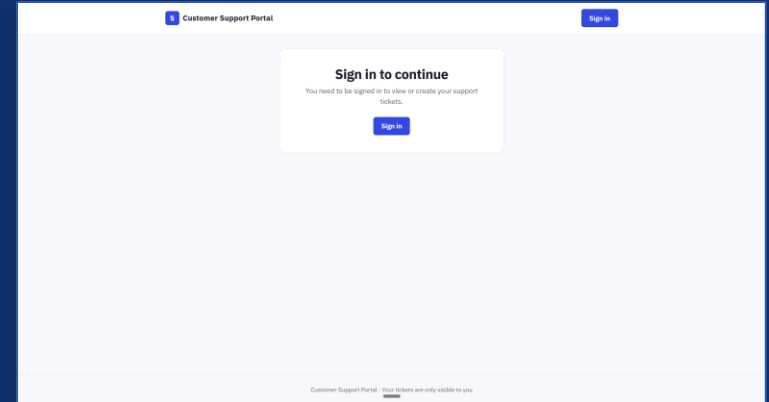
1. Deploy the site once (via PAC CLI) - creates the `.powerpages-site` folder needed for the next steps.
2. Configure authentication (choose an identity provider) so sign-in actually works.
3. Create web roles (e.g. "Authenticated Users").
4. Configure table permissions (rxrm_ticket & contact scoped to "self" - this is what actually enforces "only see your own tickets").
5. Enable Web API site settings for rxrm_ticket.
6. Redeploy so permissions/settings take effect.

How would you like to proceed?

Proceed with live environment setup? If proceeding with auth: which identity provider?

Proceed with live environment setup?:
> Yes - go ahead with all live steps (deploy, auth, roles, permissions, redeploy)
Just deploy the site for now - I'll handle auth/permissions myself later
Stop here - leave everything local, I'll deploy manually later
Other (type your answer)

↑/↓ select · enter accept · tab next · ctrl+d decline · esc cancel
```



Entra External ID

Customers self register and sign in.

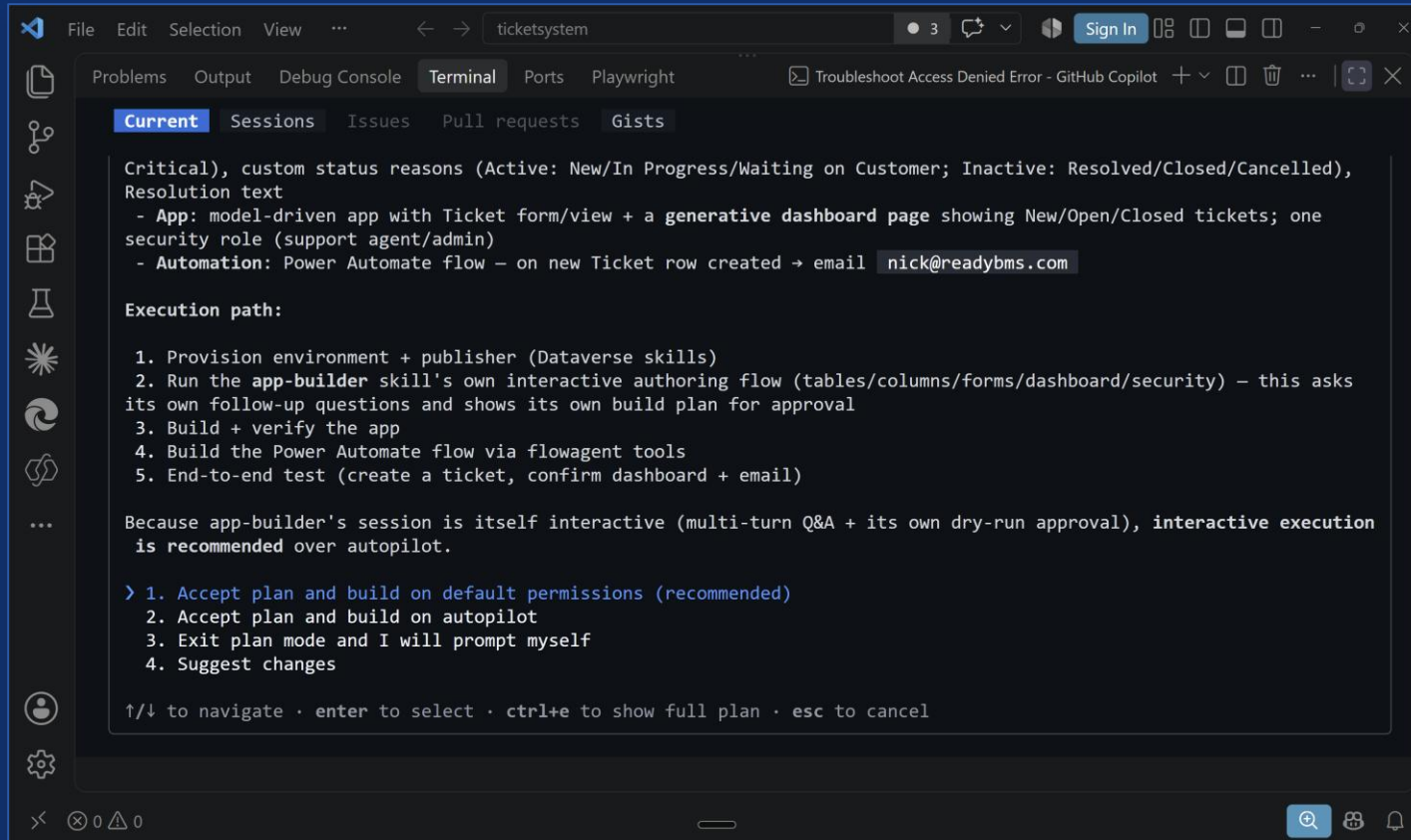
The only automatic one

The one provider the agent stands up for you, end to end.

Everything else is manual

Google, Facebook, custom OIDC: you wire those up yourself.

And then everything else



```
File Edit Selection View ... ticketsystem 3 Sign In
Problems Output Debug Console Terminal Ports Playwright
Current Sessions Issues Pull requests Gists

Critical), custom status reasons (Active: New/In Progress/Waiting on Customer; Inactive: Resolved/Closed/Cancelled),
Resolution text
- App: model-driven app with Ticket form/view + a generative dashboard page showing New/Open/Closed tickets; one
security role (support agent/admin)
- Automation: Power Automate flow - on new Ticket row created -> email nick@readybms.com

Execution path:

1. Provision environment + publisher (Dataverse skills)
2. Run the app-builder skill's own interactive authoring flow (tables/columns/forms/dashboard/security) - this asks
its own follow-up questions and shows its own build plan for approval
3. Build + verify the app
4. Build the Power Automate flow via flowagent tools
5. End-to-end test (create a ticket, confirm dashboard + email)

Because app-builder's session is itself interactive (multi-turn Q&A + its own dry-run approval), interactive execution
is recommended over autopilot.

> 1. Accept plan and build on default permissions (recommended)
2. Accept plan and build on autopilot
3. Exit plan mode and I will prompt myself
4. Suggest changes

↑/↓ to navigate · enter to select · ctrl+e to show full plan · esc to cancel
```

What should it look like?

Minimal and clean, bold and vibrant, or dark and moody.

How many severity levels?

A real data model decision, asked in plain language.

Dry run or build?

It offers to walk the plan before it writes anything.

Answer properly

A lazy answer now is a rebuild later.

Answer from your phone

Hand the work to the GitHub Copilot coding agent and it keeps going on github.com. You review and reply from the GitHub mobile app.

1 Delegate the issue

Assign the work to Copilot from the repo. It opens a pull request and starts.

2 It reports as it goes

Progress, commits and questions land in the pull request thread.

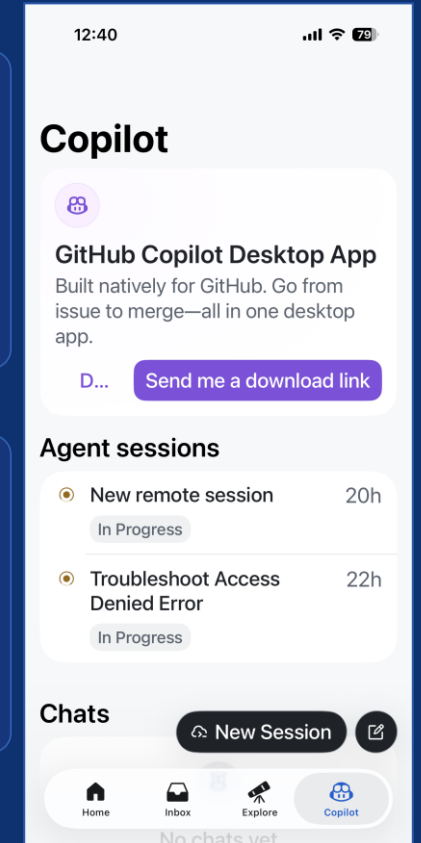
3 You answer from anywhere

Comment from your phone. The agent picks it up and carries on.

4 You still review the diff

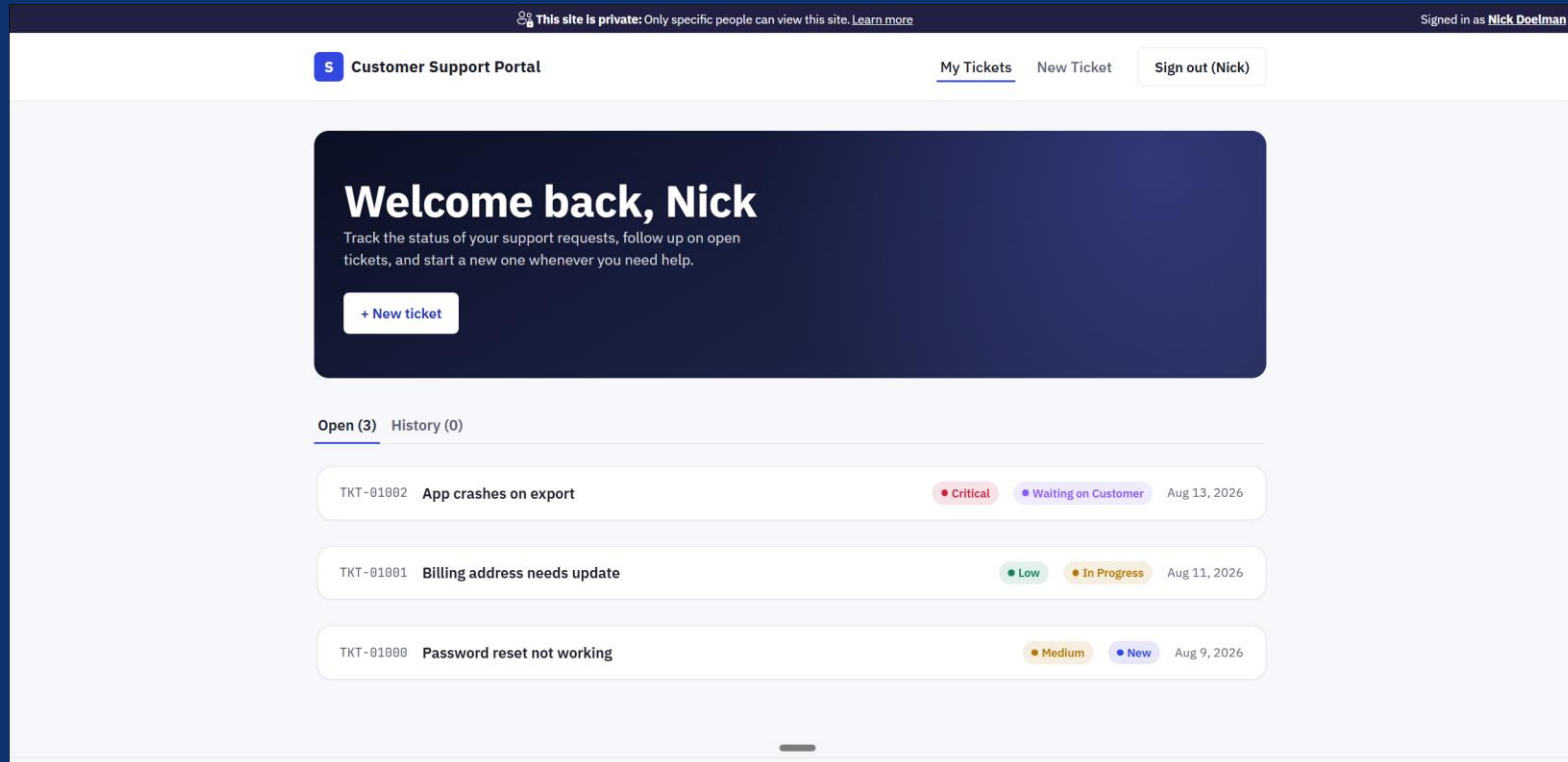
Nothing merges because it felt confident. You merge it.

The honest version: this works well for contained changes. A full site build still wants you at a keyboard.



THE RESULT

What came out the other end



A conversation became a secure, data driven site. It also took longer than the demo makes it look.

6

Where it breaks

The honest part. Real builds hit real walls.

Table permissions

The agent still does not get this right. Plan to do it yourself.

Customer Support Portal

My Tickets New Ticket Sign out (Nick)

New Ticket

Tell us what's going on and we'll get back to you.

You don't have permission to associate or disassociate table contact to `new_ticket`

Topic
test

Description
test

Severity
High

Date
2026-08-19
Defaults to today -- change it if this happened on another day.

Submit ticket

Customer Support Portal

My Tickets New Ticket Sign out (Nick)

New Ticket

Tell us what's going on and we'll get back to you.

You don't have permission to associate or disassociate table contact to `new_ticket`

Topic
test

Description
test

Severity
Medium

Date
2026-08-19
Defaults to today -- change it if this happened on another day.

Submit ticket

Customer Support Portal

My Tickets New Ticket Sign out (Nick)

New Ticket

Tell us what's going on and we'll get back to you.

An unexpected error occurred while processing the request

Topic
test

Description
test

Severity
Medium

Date
2026-08-19
Defaults to today -- change it if this happened on another day.

Submit ticket

Three attempts. Three permission errors on the same form.

What goes wrong

It creates the permission but scopes it wrong, or misses the child table on the form.

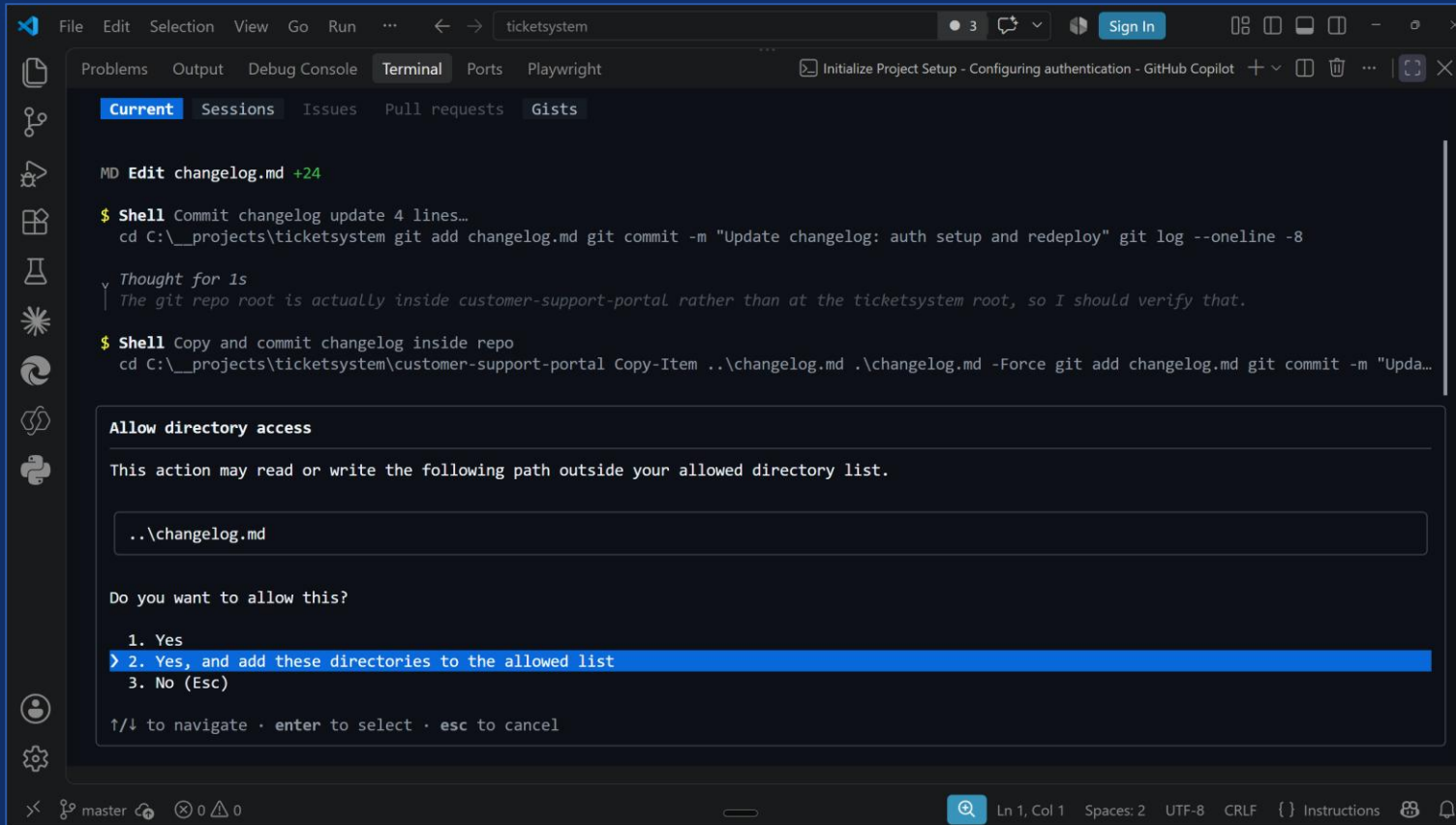
What to do

Set the permission yourself in the design studio, then tell the agent it is done.

Why it matters

This is the control that keeps one customer from reading another one's tickets. Check it by hand.

Unblocking the .js files



The screenshot shows a VS Code terminal window with the following content:

```
MD Edit changelog.md +24
$ Shell Commit changelog update 4 lines...
cd C:\_projects\ticketssystem git add changelog.md git commit -m "Update changelog: auth setup and redeploy" git log --oneline -8
Thought for 1s
The git repo root is actually inside customer-support-portal rather than at the ticketssystem root, so I should verify that.
$ Shell Copy and commit changelog inside repo
cd C:\_projects\ticketssystem\customer-support-portal Copy-Item ..\changelog.md .\changelog.md -Force git add changelog.md git commit -m "Upda...
```

Below the terminal output, a security warning is displayed:

Allow directory access

This action may read or write the following path outside your allowed directory list.

..\changelog.md

Do you want to allow this?

1. Yes
2. Yes, and add these directories to the allowed list
3. No (Esc)

↑/↓ to navigate · enter to select · esc to cancel

An environment level setting. Know what you are changing before you say yes.

The symptom

The site deploys, then the browser gets nothing back for the script files.

The cause

Power Pages blocks .js as an attachment type by default, at the environment level.

The fix

Change blockedattachments in the environment settings. The agent asks first.

WHERE IT BREAKS

Debug it together

The fastest fix is showing the agent what you are looking at.

This screenshot shows a web interface for a 'Customer Support Portal'. At the top, there's a navigation bar with 'My Tickets', 'New Ticket' (active), and 'Sign out (Nick)'. The main heading is 'New Ticket' with a subtext 'Tell us what's going on and we'll get back to you.' Below this is a form with four fields: 'Topic' (containing 'test'), 'Description' (containing 'test'), 'Severity' (a dropdown menu set to 'High'), and 'Date' (set to '2026-08-19'). A red error message is displayed at the top of the form: 'You don't have permission to associate or disassociate table contact to rxrm_ticket'. At the bottom of the form is a 'Submit ticket' button.

Real builds hit real walls. Showing this on stage is the point, not the risk.

1 Paste the screenshot

Drop the error straight into the chat. It reads it.

2 Share the browser

Give it the DevTools network tab and the failing request.

3 Ask for the cause

Not "fix it". Ask why it broke, then approve the fix. Log it.

Ask it to write the spec it wishes it had

Ask what it missed

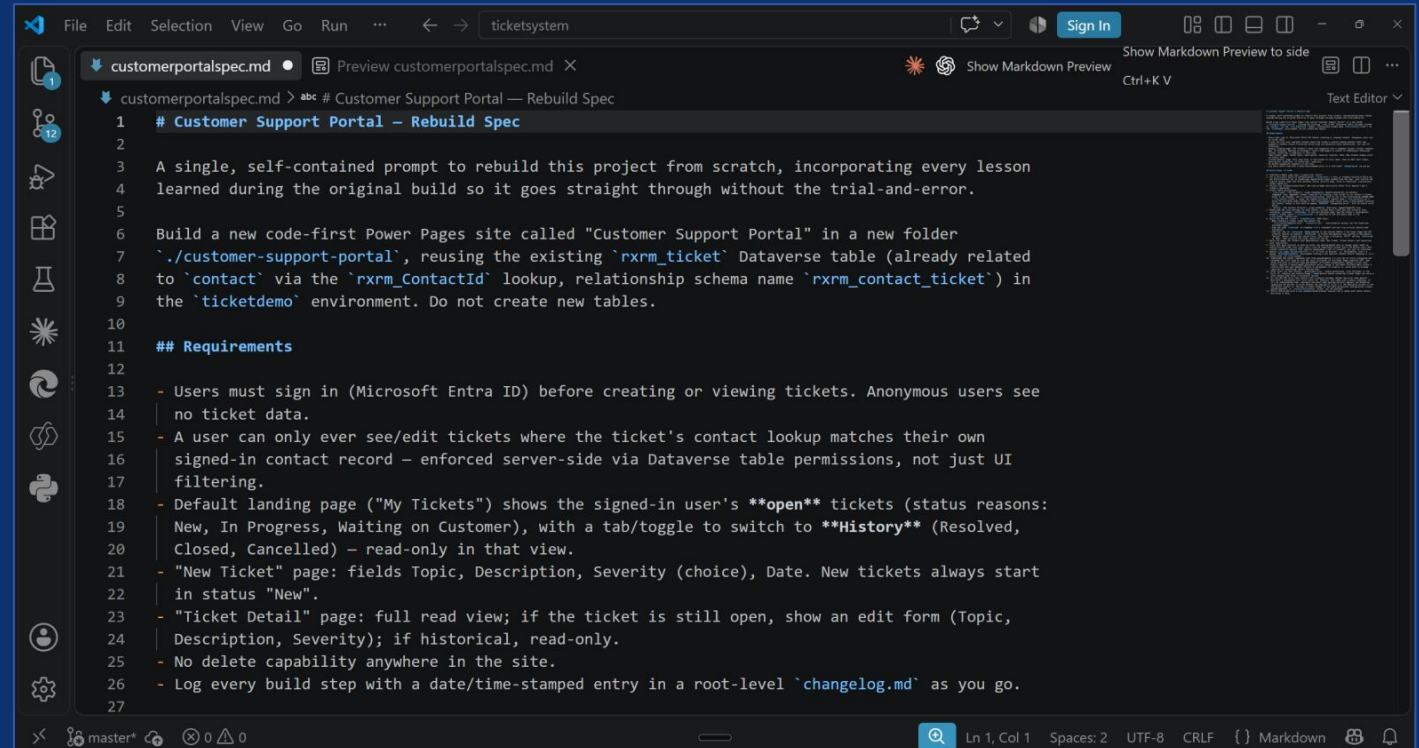
Point the agent at the finished project and the changelog. It tells you what it knows now that it did not on day one.

Fold it into one prompt

Tables, permissions, the blocked .js setting, the order of operations. One spec that goes straight through.

Throw the build away

The code was always the cheap part. The expensive part was the learning, and the spec keeps all of it.



```
customerportalspec.md > abc # Customer Support Portal — Rebuild Spec
1 # Customer Support Portal — Rebuild Spec
2
3 A single, self-contained prompt to rebuild this project from scratch, incorporating every lesson
4 learned during the original build so it goes straight through without the trial-and-error.
5
6 Build a new code-first Power Pages site called "Customer Support Portal" in a new folder
7 './customer-support-portal', reusing the existing 'rxrm_ticket' Dataverse table (already related
8 to 'contact' via the 'rxrm_ContactId' lookup, relationship schema name 'rxrm_contact_ticket') in
9 the 'ticketdemo' environment. Do not create new tables.
10
11 ## Requirements
12
13 - Users must sign in (Microsoft Entra ID) before creating or viewing tickets. Anonymous users see
14 no ticket data.
15 - A user can only ever see/edit tickets where the ticket's contact lookup matches their own
16 signed-in contact record — enforced server-side via Dataverse table permissions, not just UI
17 filtering.
18 - Default landing page ("My Tickets") shows the signed-in user's **open** tickets (status reasons:
19 New, In Progress, Waiting on Customer), with a tab/toggle to switch to **History** (Resolved,
20 Closed, Cancelled) — read-only in that view.
21 - "New Ticket" page: fields Topic, Description, Severity (choice), Date. New tickets always start
22 in status "New".
23 - "Ticket Detail" page: full read view; if the ticket is still open, show an edit form (Topic,
24 Description, Severity); if historical, read-only.
25 - No delete capability anywhere in the site.
26 - Log every build step with a date/time-stamped entry in a root-level 'changelog.md' as you go.
27
```

Rebuilding is not starting over

Agentic tools made the code the cheap part. Keep the spec, keep the lessons, and regenerate the site instead of patching it.

CHOOSING

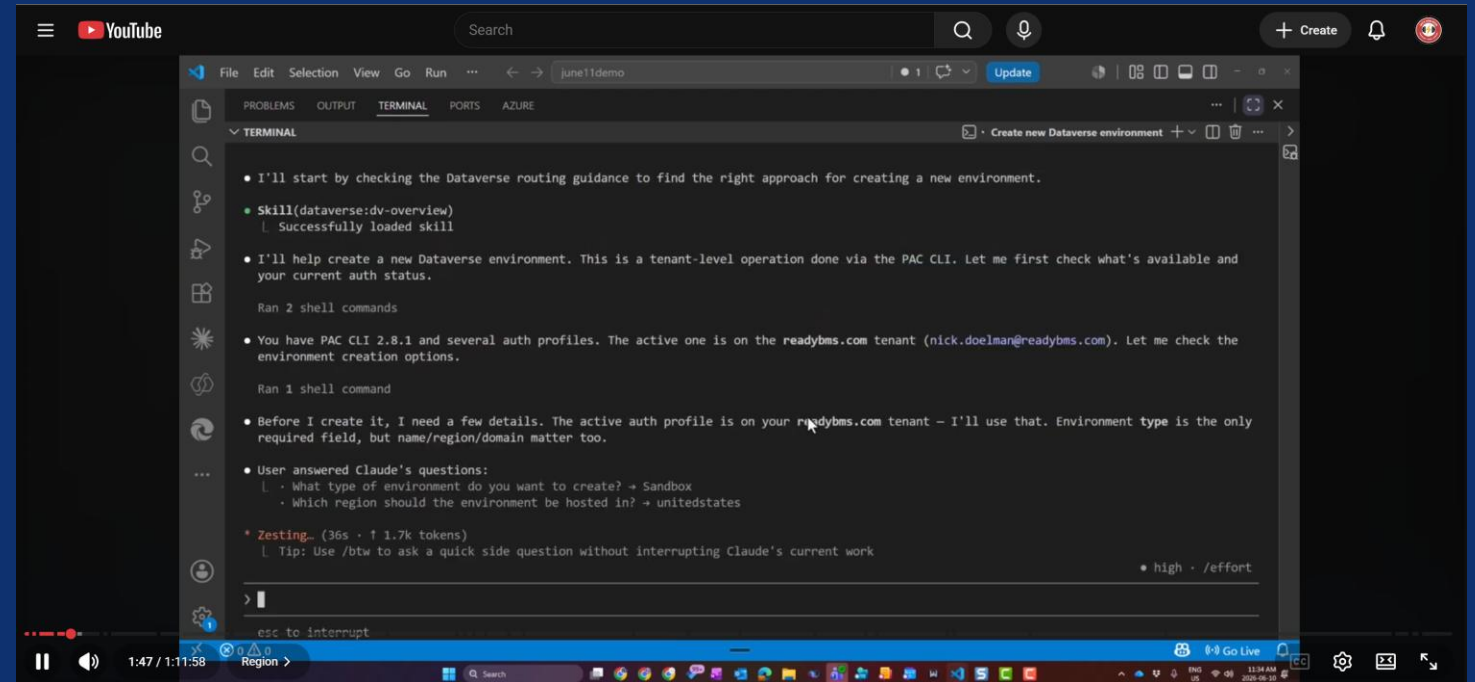
Regular Power Pages against code first

	Regular Power Pages	Code first Power Pages
Time to first version	Fast for standard lists and forms	Fast now that an agent writes the code
Who can change it later	Makers, in the browser	Developers, in Visual Studio Code
Interaction ceiling	Page reloads, platform patterns	Anything React can do
What you need to know	Liquid, web templates, configuration	Node, React, a build step, the PAC CLI
Debugging	Portal tooling and Liquid errors	Browser DevTools and the Web API
Keeping it current	Microsoft moves the platform forward	You own the dependencies and the upgrades
Where it fits	Forms over data, content, self service	Dashboards, rich interaction, app-like sites

Neither column is the right answer. Pick the row that describes your project and read across.

Agentic coding end to end process to build a site

<https://youtu.be/dpY5WsXg1Qk>



Copy of the presentation

<https://github.com/readyxrm/HelishSummit2026>



Kiitos, Helsinki!

Plan first

The agent is fast at typing. It is not fast at guessing your data model.

Skills, not just an agent

Dataverse and Power Platform skills are what make the output usable.

Check the security yourself

Table permissions are still yours to get right.

Configured is still fine

Code first is a choice, not an upgrade.

[Rate this session](#)

Nick Doelman

readyxrm.blog · nickdoelman.com · [Power Platform BOOST podcast](#)

Questions?



🌟 Huge thanks to our awesome sponsors 🌟

Headline sponsor



Platinum sponsors



Gold sponsors



Silver sponsors





Kiitos!